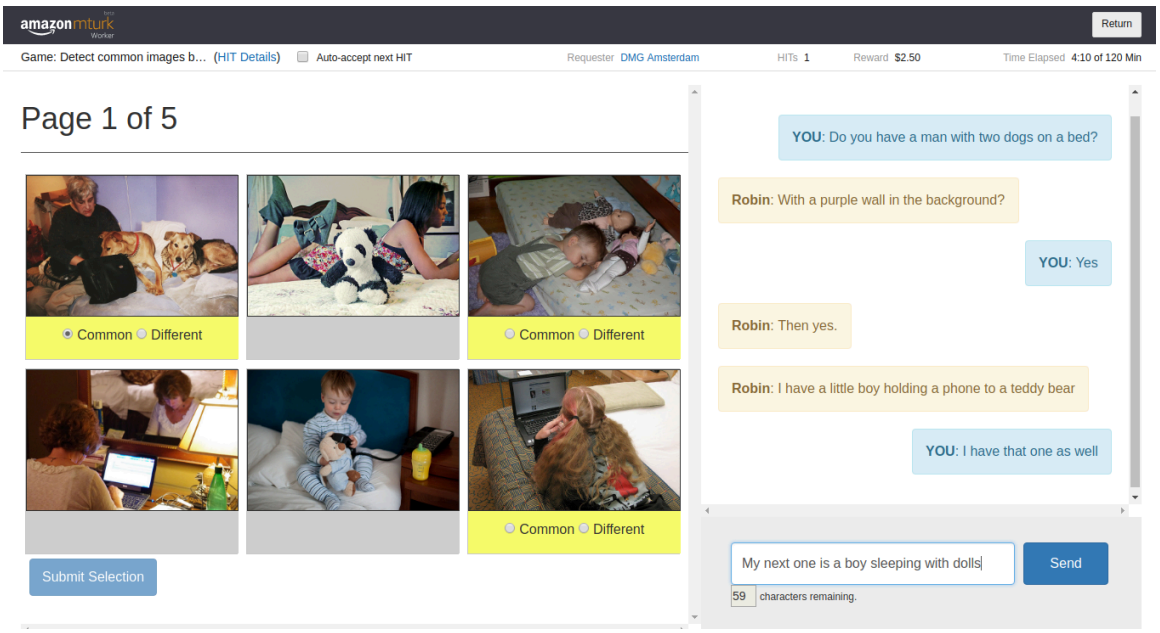
Artificiell intelligens: kognitiva system - Report

How does robot human interaction look like? How do we imagine communication with robots in a household environment? In my opinion the goal is to achieve the level of communication where it will be possible to manage robots with the help of dialogue, which seems to be the most natural way to instruct another being by a human. In the past few years the large language model achieved very good results in a short period of time. But together with LLMs we observed the development of visual language models. VLMs are not yet on as advanced a stage as LLMs and require more research. VLM is important in the *cognitive systems* field since the robot works in a multimodal environment. For example a well functioning household robot needs to take visual input, process it and be able to have conversation based on that input. This is called visually grounded dialogue. All sorts of issues that this topic brings are being researched nowadays.



In my experiment I was testing different tools and models to check if machine learning models are able to detect which image from the data set is being described by a particular utterance. I was working with a photobook dataset(Haber 2019). This dataset was created based on the game. In the game two players are presented with two different pages of a photobook. Each photobook page contains 6 images. Some of those images overlap between players, and some are different. The task for the players is to detect which images are common and which are different. They communicate with each other via chat. This dataset allows one to choose a focus point for research. For example it is possible to either focus on conversation level and the interaction between players in context of photobook

pages, or the other focus can be on particular images, and how they were described by the players. In my experiment I'm focusing on the relation between a particular photo (not the whole photobook page) and particular utterances that were being used to describe this image.

As my baseline model I created a contrastive matching model. I preprocessed the data from the Photobook dataset. I transformed the images with help of CV2 library and, text descriptions were encoded with simple character to index transformation. The issue of the data set was that one image had few descriptions, so instead of creating one image description pair, I created multiple pairs with the same image. Probably the safer approach would be to choose just one description for each image, but the dataset wasn't big enough for that. I split the data in 8/1/1 ratio for train, dev, and test set accordingly. I used the same ratio in other experiments as well. I used the Dataset class from pytorch to prepare data for training. The *collate_fn* function that is built in the Dataset class took care of padding. The neural network architecture is composed of two main elements. Image encoder and Text encoder. For the Image encoder I used predesigned res-net architecture *resnet18* that is part of pytorch library(Pytorch 2017). For the text encoder I also used a predesigned transformer(Pytorch 2017b). The last line of the forward function returns the logit function. It multiples the text embedding matrix with transposed image embedding matrix and divides it by the temperature to create the similarity matrix. The temperature is just predecided value(0.07 in my case), that scales the values to be more determined.

```
logits = txt_emb @ img_emb.T / self.temperature
return logits
```

In the training function the novelty is the loss function. It is a contrastive loss function used for contrastive matching models. This loss function counts the cross entropy between blank matrix and logit matrix for both combinations: image-text, and text-image. The returned value is just an average of both these loss values.

```
def contrastive_loss(logits):
    B = logits.size(0)
    labels = torch.arange(B, device=logits.device)
    loss_t2i = F.cross_entropy(logits, labels)
    loss_i2t = F.cross_entropy(logits.T, labels)
    return (loss_t2i + loss_i2t) / 2
```

The loss function is being fed back to the model with the *Adam* optimizer.

Since it is the contrastive matching model the accuracy of the model is not tested with the top match, but often with the range of top matches. I tested the model for the matching description in top 5 matches. The baseline model did very poorly with 0.34 accuracy.

For the second model experiment I tried to test if using some pretrained model to process image input will help with the accuracy. For this purpose I chose meta's detectron2 tool(Yuxin 2019). Detectron2 is a tool containing lots of model variations that are specialised in object detection and segmentation. I used the zoo model and ran it through Photobook dataset's images.

```
0 - Oriental girl decorating a red umbrella.
1 - Do you have the girl decorating the red umbrella?
2 - What about the girl decorating the red umbrella?
```

This model provided me with predicted objects placement and predicted classes from the Coconet classes set. I decided to focus on names of classes of predicted objects for each image. I reused the architecture from the baseline model, but this time instead of images, I provided the model with a list of detected objects. Because of text/text input as opposed to image/text input I used textencoder for both inputs. In my first vision this model was supposed to work better than the baseline model, it had worse accuracy than the baseline model around 0.31. With this failure I decided to give it one more try but this tim with fine tuning a well known model.

For the last experiment I chose to finetune the CLIP model (Cherti 2023). Clipmodel is a contrastive matching model released by Open AI. So the same type of model I was trying to design myself in the first two experiments. I fed the model with the same images and descriptions as in the first experiment but this time I wasn't embedding them myself but I used the provided transforming functions from a particular model(ViT-B-32).

Since the third experiment is just fine tuning I did not design neural network architecture myself, but only a training loop. The training loop starts with pytorch's *functional.normalize* function that ensures cosine similarity will be possible between 2

different embedding spaces. And also the temperature was set by me in the training loop since I couldn't control the nn architecture. The rest of the training is the same as in previous experiments. Unfortunately even though my goal was to finally have some results, the fine tuned CLIP model has the same accuracy as previous models.

Even though my first plan was to find the best method with these three experiments I found out that this part of the data set is not suitable for contrastive-matching modeling. The descriptions were not descriptions per se, but were part of the bigger conversation. Hence none of the models worked. The other aspects of this dataset are promising, and I hope to get better results with my bachelor thesis.

Stanislaw Lorys

Bibliography:

Cherti, M., Beaumont, R., Wightman, R., Wortsman, M., Ilharco, G., Gordon, C., Schuhmann, C., Schmidt, L., & Jitsev, J. (2023). Reproducible scaling laws for contrastive language-image learning. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 2818–2829).

Haber, J., Baumgärtner, T., Takmaz, E., Gelderloos, L., Bruni, E., & Fernandez, R. (2019). The PhotoBook Dataset: Building Common Ground through Visually-Grounded Dialogue. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (pp. 1895–1910). Association for Computational Linguistics.

Pytorch 2017 - https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html

Pytorch 2017b -https://docs.pytorch.org/docs/stable/generated/torch.nn.TransformerEncoder.html

Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo, & Ross Girshick. (2019). Detectron2. .